

ECM3412/ECMM409
Nature Inspired Computation
Lecture 8

Ant Colony Algorithm Detail and
Applications

The Traveling Salesperson Problem: A reminder

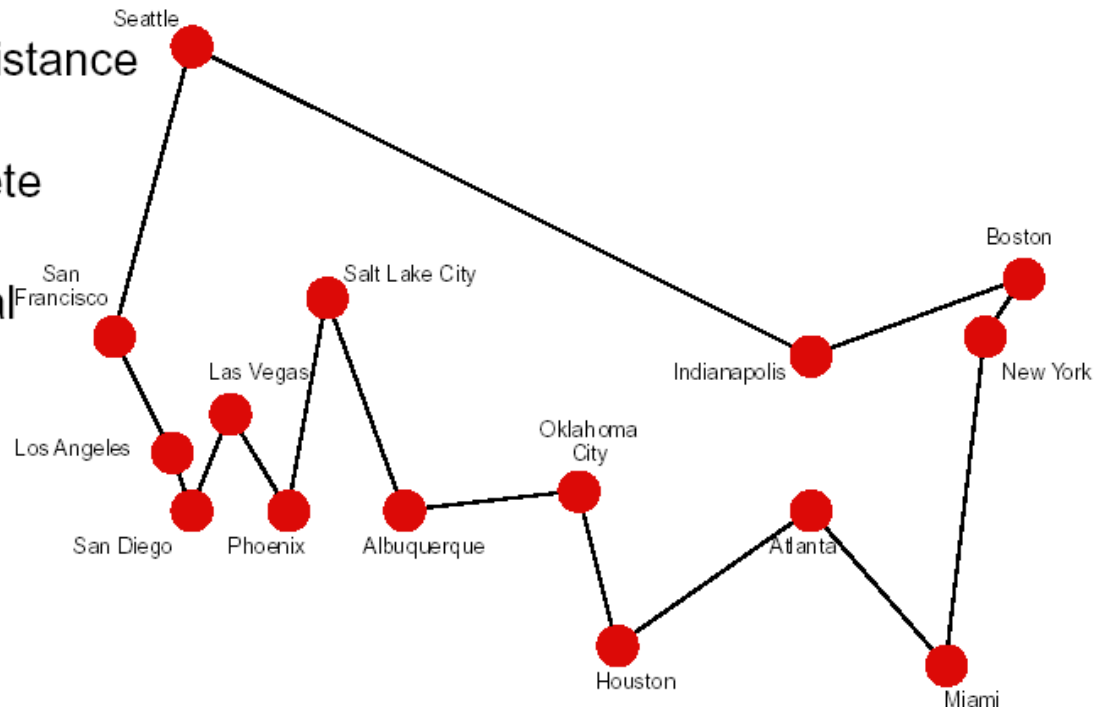
Problem: given N cities, and a distance function d between cities, find a tour that:

(1) goes through every city once and only once

(2) minimizes the total distance

■ Problem is NP-complete

■ Classical combinatorial optimization problem to test algorithms



The *Ant System* Algorithm

Loop

Randomly position *num_ant* ants on *num_city* cities

For step := 1 to *num_city*

For k := 1 to *num_ant*

Choose the next city to move to applying a
probabilistic ***state transition rule***

End-for

End-for

Update pheromone trails

Until End_condition

About the transition rule

For each ant the transition from city i to city j at iteration t depends on:

- Has the city been visited? Each ant k maintains a tabu list in memory that defines the set J_i^k cities still to be visited when at city i .
- A (static) local heuristic $\eta_{ij} = 1/d_{ij}$ that represents the desirability of visiting city j when in city i .
- The (learned, global) amount of virtual pheromone on trail $(i,j) = \tau_{ij}(t)$.

The Transition Rule

- The probability for ant k to go from city i to city j while in its t^{th} tour is:

$$p_{ij}^k(t) = \frac{[\tau_{ij}(t)]^\alpha \cdot [\eta_{ij}(t)]^\beta}{\sum_{h \in J_i^k} [\tau_{ih}(t)]^\alpha \cdot [\eta_{ih}(t)]^\beta}$$

- If j has been visited then $p() = 0$
- Note α β are parameters – trades off global vs local information
- If α is small then the closest cities are favored – classic greedy algorithm
- If β is small only pheromone +ve feedback at work – may choose non-optimal paths too quickly

The Update Rule

- After the completion of a tour each ant k lays a quantity of pheromone on each edge (i,j) that it has used.
- The amount is equal to Q/L (fitness)
- Also, Pheromone evaporates: $\tau \rightarrow (1-\rho) \tau$
- Rule:
$$\tau_{ij}(t+1) \leftarrow (1-\rho) \cdot \tau_{ij}(t) + \Delta\tau_{ij}(t)$$

where

$$\Delta\tau_{ij}(t) = \sum_{k=1}^n Quality^k$$

$Quality^k$, in the TSP case, can be set to the inverse of the length of the solution found by ant k

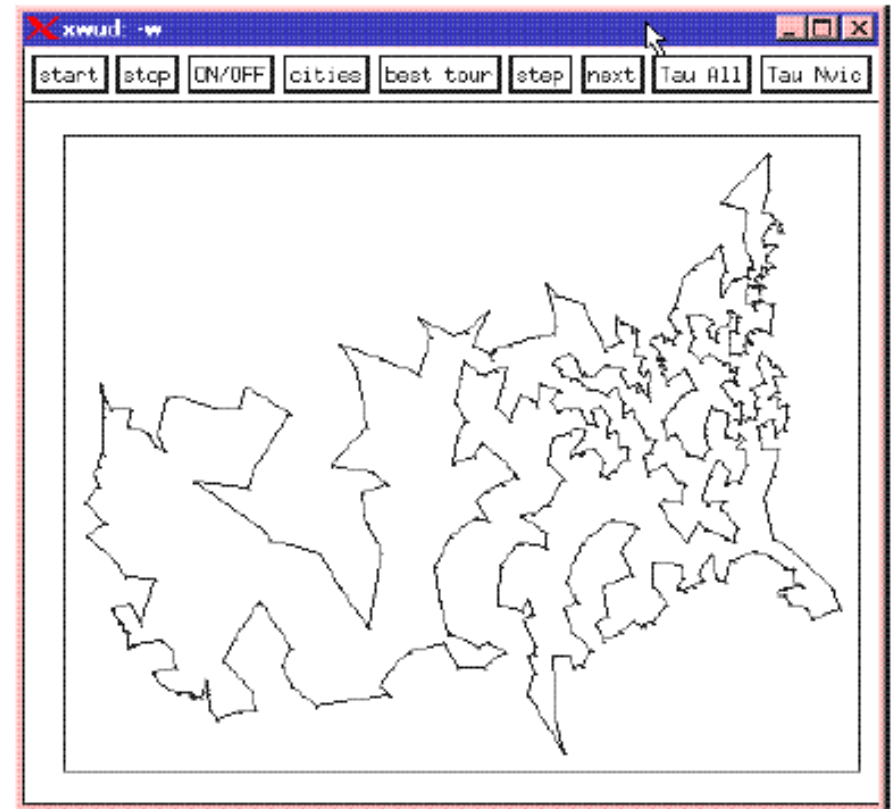
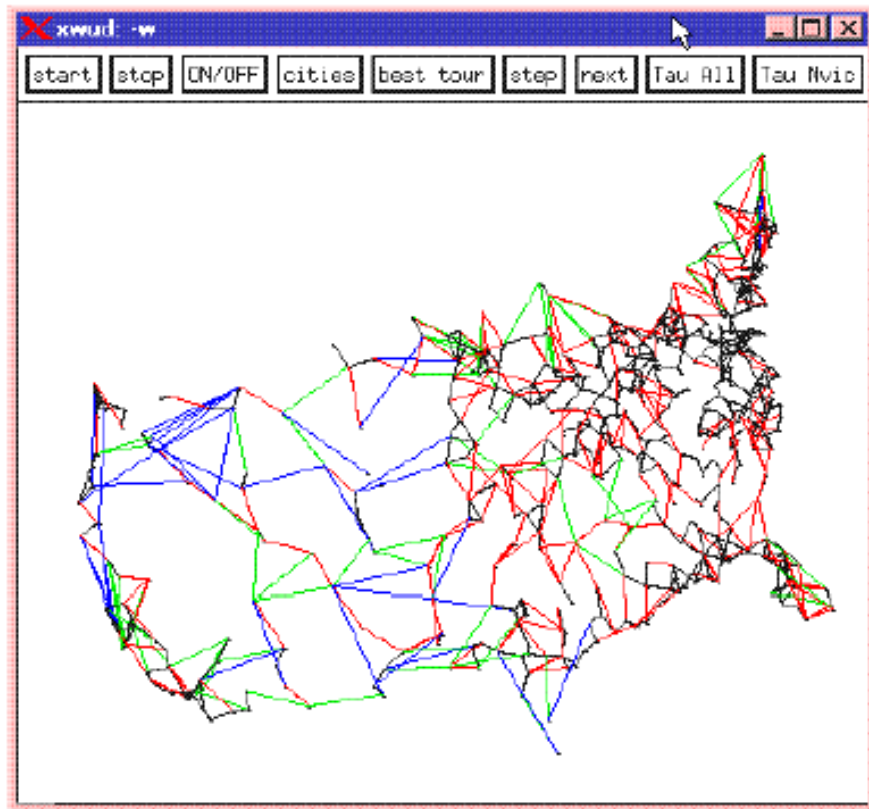
Early Ant Results

- Algorithm found best solutions on small problems (30 city)
- On larger problems converged to good solutions – but not the best
- On “static” problems like TSP hard to beat specialist algorithms
- Ants are “dynamic” optimizers – should we even expect good performance on static problems
- Coupling ant with local optimizers gave world class results....

ACS ran for 1250 iterations (end criterion) using 20 ants; average over 15 runs for all implementations

Problem	Ant Colony System (Dorigo & Gambardella, 1997)		Genetic Algorithms (Whitley et coll., 1989)		Simulated Annealing (Lin et coll., 1993)	
	Shortest Tour Length	Number of tours	Shortest Tour Length	Number of tours	Shortest Tour Length	Number of tours
Eil50 (50 cities)	425	1830	428	25000	443	68512
Eil75 (75 cities)	535	3480	545	80000	580	173250
KroA100 (100 cities)	21282	4820	21761	103000	N/A	N/A

An Example Ant Colony Optimisation:



Extensions to the Ant Colony Optimisation Algorithm

- The original system by Dorigo is known as the Basic Ant System (AS) (1996)
- There are other variants that have been found to perform better in some problems:
 - Max-Min Ant System (Stutzle & Hoos, 2000)
 - Elitist Rank Ant System (Bullenheimer, 1999)
- These systems update the pheromone trails differently – using only the best or the best n ants to update the trails

Not just for TSP of course

ACO is naturally applicable to any sequencing problem, or indeed *any* problem

All you need is some way to represent solutions to the problem as paths in a network (construction graph).

E.g.

Single machine scheduling with due-dates

These jobs have to be done; their length represents the time they will take.

A

B

C

D

E

E.g.

Single machine scheduling with due-dates

These jobs have to be done; their length represents the time they will take.

Each has a 'due date', when it needs to be finished

A	3pm
B	3:30pm
C	5pm
D	4pm
E	4:30pm

E.g.

Single machine scheduling with due-dates

These jobs have to be done; their length represents the time they will take.

Each has a 'due date', when it needs to be finished



A

3pm



B

3:30pm



C

5pm



D

4pm



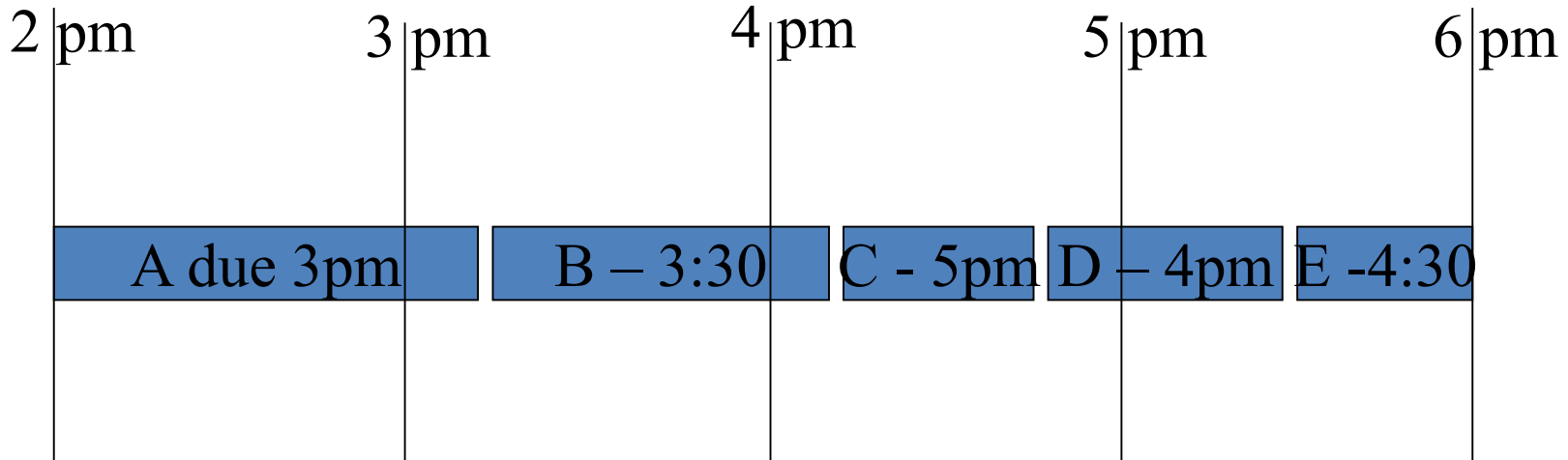
E

4:30pm

Only one 'machine' is available to process these jobs, so can do just one at a time.

[e.g. machine might be human tailor, photocopier, Hubble Space Telescope, Etc ...]

An example schedule



A is 10min late

B is 40min late

C is 20min early (lateness = 0)

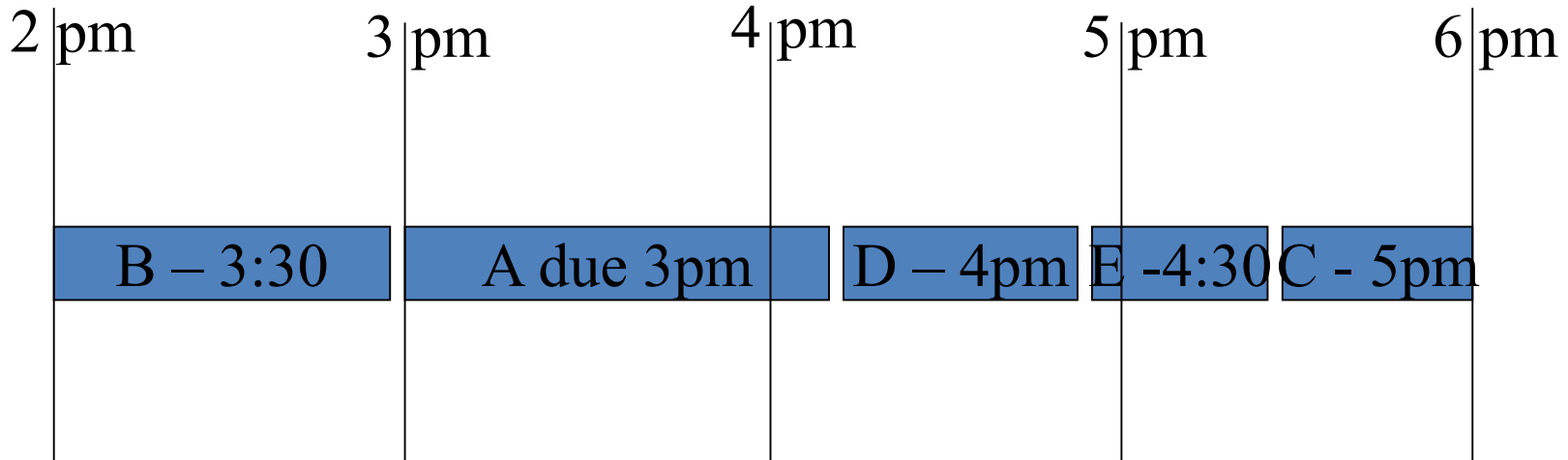
D is 90min late

E is 90min late

Fitness might be average lateness;
in this case 46min

or fitness could be Max lateness,
in this case 90min

Another schedule



A is 70min late

B is 30min early (0 lateness)

C is 60min late

D is 50min late

E is 50min late

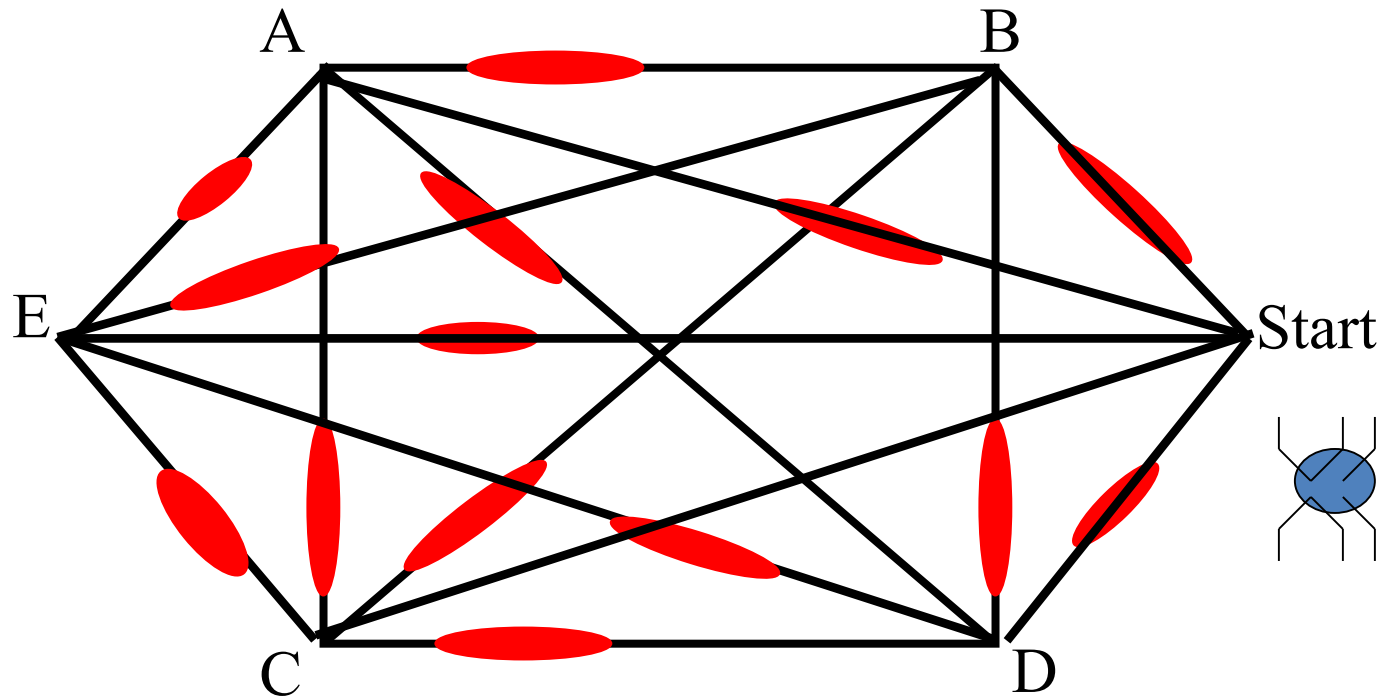
Fitness might be average lateness;
in this case again 46min

or fitness could be Max lateness,
in this case 70min

Applying ACO to this problem

Just like with the TSP, each ant finds paths in a network, where, in this case, each job is a node. Also, no need to return to start node – path is complete when every node is visited.

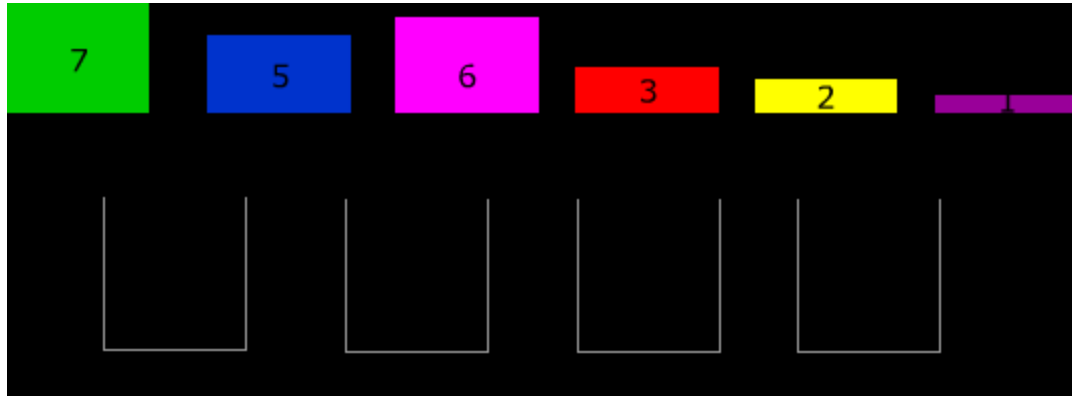
Initially, random levels of pheromone are scattered on the edges, an ant starts at a *Start* node (so the first link it chooses defines the first task to schedule on the machine); as before it uses a transition rule to take one step at a time, biased by pheromone levels, and also a heuristic score, each time choosing the next machine to schedule. What heuristic might you use in this case?



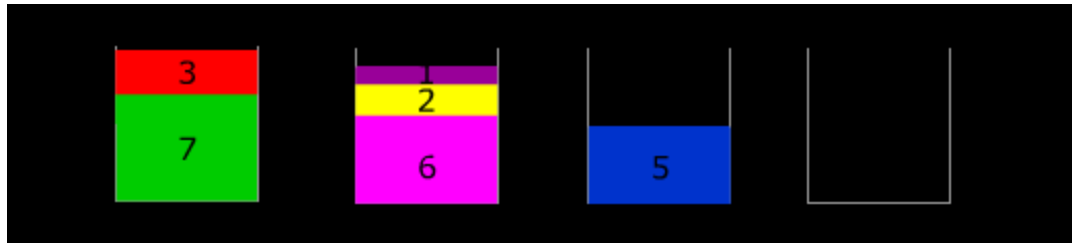
Why is it sensible to have a Start node for this problem

Bin Packing (CA)

K items (6)
B bins (4)

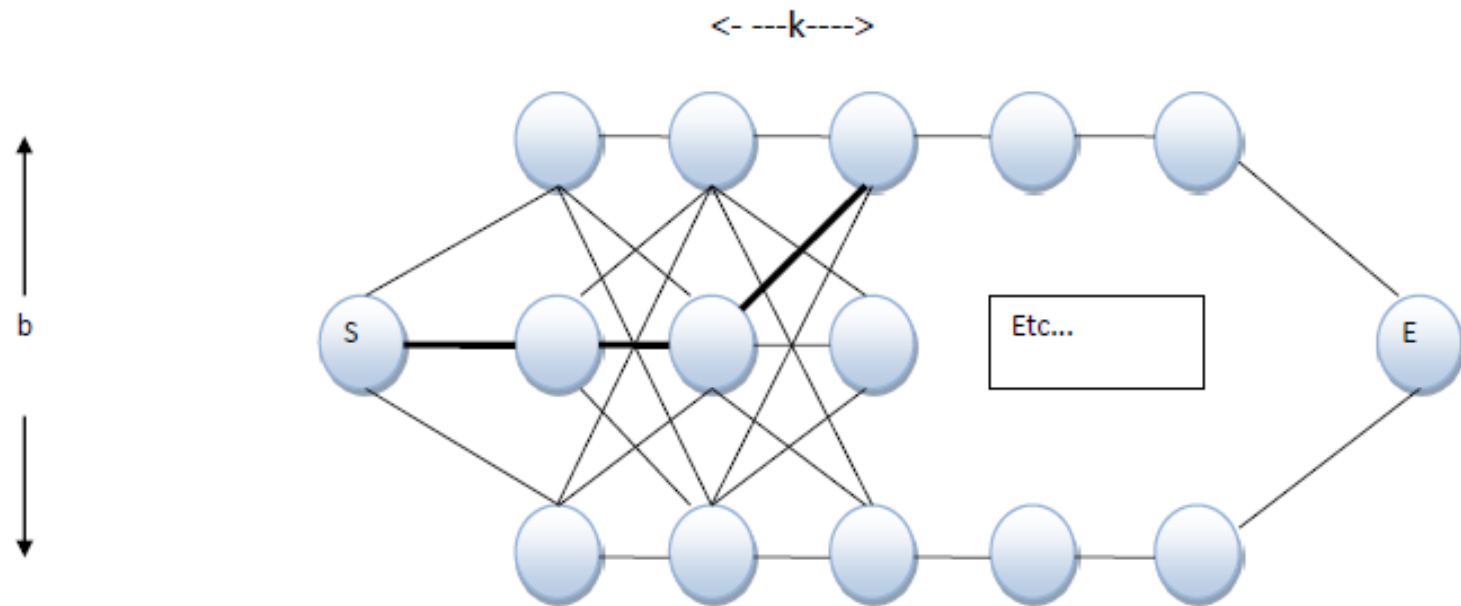


Objective:
minimising
difference in
weight



Construction Graph for CA

Ants can move only forward one layer each step



3 bins
5 items

item 1 → bin 2
item 2 → bin 2
item 3 → bin 1
...

Conclusions

- Ant colony main components:
 - Transition Rule
 - Update Rule
- Discrete optimisation problems (other than TSP) can be optimised using a construction graph
- Results are often competitive with evolutionary algorithms